

FoundIt — Viva Preparation

For the three things you were asked: **individual contributions**, **how it works**, and **show the code and explain it**.

Ownership below is taken from the `OWNER:` tags in the source, not invented. Correct anything that does not match what actually happened — an examiner can tell when someone is describing code they did not write.

0. The awkward question, answered honestly

`git shortlog -sn` shows roughly 55 commits from Nidhi and 6 across the other three. If the examiner looks, do not talk around it. Something like:

"We paired on most of the work and pushed from Nidhi's machine, so the commit history under-represents the others. The ownership is real though — each of us owns and can explain our modules, and that's marked in the source."

Then let each person prove it by explaining their own code. That is the only convincing answer, and it is why the sections below matter.

If that is *not* what happened, say what did. A truthful "I wrote most of it and they contributed X, Y, Z" costs less than a story that unravels.

1. Who owns what

Nidhi — data model, feed, matching

File	Role
<code>web/src/types.js</code>	The frozen contract. Every collection shape; imported by frontend and functions
<code>web/src/lib/feed.js</code>	Feed reads/writes, the denormalized card mapping
<code>web/src/lib/matching.js</code>	Client-side four-factor match scorer
<code>functions/index.js</code> → <code>suggestMatches</code>	Server-side matching trigger
<code>functions/src/scoring.js</code>	Match score maths

Be able to explain: why the feed denormalizes poster fields onto each item instead of joining; how the match score is composed; why `matchScore` is server-owned.

Shanid — authentication, trust, roles

File	Role
<code>web/src/auth/authApi.js</code>	Google sign-in, campus domain enforcement, profile bootstrap
<code>web/src/auth/AuthContext.jsx</code>	Auth state, live profile subscription
<code>functions/index.js</code> → <code>recomputeTrustScore</code> , <code>setUserRole</code> , <code>resolveFlag</code>	Trust + RBAC
<code>functions/src/scoring.js</code>	Bayesian trust maths

Be able to explain: why the `hd` parameter is not security; the damping term in the trust formula; how custom claims reach the security rules.

Hadi — infrastructure, notifications

File	Role
<code>web/src/lib/firebase.js</code>	SDK init, emulator wiring, the prod/dev guard
<code>web/src/lib/notifications.js</code>	Notification read model
<code>firebase.json</code> , <code>vercel.json</code>	Emulator + deployment config

Be able to explain: why the emulator connect is guarded on `import.meta.env.DEV`; why clients can read notifications but never create them.

Shenza — UI, card shape, interaction flows

File	Role
<code>web/src/components/ItemCard.jsx</code>	The card the whole feed renders
<code>web/src/lib/feed.js</code> (card mapping)	Firestore doc → card shape
<code>web/src/components/</code> modals	Claim, chat, deal, flag, smart match
<code>web/src/index.css</code>	The 642-line design token system, dark mode

Be able to explain: why the card shape is a mapping layer rather than raw Firestore documents; how dark mode works through CSS custom properties.

2. How it works — the 90-second version

"A student posts a lost or found report. It's written straight to Firestore from the client, because the rules enforce who may write what. That write fires a Cloud Function that scores it against every complementary report and writes back a match. Every other client is subscribed to the same collection over a snapshot listener, so the new item and its match badge appear without anyone refreshing. Claiming is a two-party handshake; trust is recomputed server-side after every resolved exchange, and clients are physically prevented from writing their own trust score."

Four ideas underneath that, and an examiner will probe each:

1. **Event-driven** — writes emit events; functions react. No polling anywhere.
2. **Server-authoritative state** — trust, status and match scores are computed only in functions, and the rules reject client writes to those fields.
3. **Read model** — poster name, department and trust are copied onto each item so the feed renders with no joins. Firestore has no joins by design.
4. **Graceful degradation** — AI, matching and images all have fallbacks. The app never hard-fails on an external dependency.

3. Code walkthrough — what to open and what to say

Have these four files open in tabs before you start.

3a. The security rule that makes trust trustworthy

firestore.rules

```
function keeps(f) { return request.resource.data[f] == resource.data[f]; }

match /users/{uid} {
  allow read: if signedIn();
  allow update: if signedIn() && request.auth.uid == uid
    && keeps('trustScore') && keeps('trustTier') && keeps('role')
    && keeps('ratingAvg') && keeps('ratingCount') && keeps('strikes');
}
```

"You may edit your own profile, but the update is rejected unless every server-owned field is unchanged. So a user can rename themselves and cannot touch their trust score or role. Functions use the Admin SDK, which bypasses rules — that's the only path that can write these."

Expect: "Could a user just call the function instead?" → `setUserRole` checks `req.auth.token.role == 'admin'` from the ID token, which is signed by Firebase and not client-editable.

3b. Trust scoring

functions/src/scoring.js

```
(avg - 3) * (count / (count + 5)) * 40
```

"Deviation from neutral, damped by review count. With one five-star review the damping factor is 1/6, so you get about 13% of the possible swing. It takes roughly 15 consistent reviews to approach the full range — that's what stops a new account farming a few reviews from friends."

Expect: "Why 5?" → It's the prior weight; larger means slower to trust. Tuned by hand, not learned.

3c. Matching

`web/src/lib/matching.js` and `functions/index.js` → `suggestMatches`

Four weighted factors: category 30, keyword overlap 40, location 20, recency 10.

Run the demo pair live if asked — lost "blue spiral lab record notebook" against found "blue spiral lab record book in LH-101" scores **99**, breaking down as 30 category + 40 keywords (blue, spiral, lab, record, 101) + 20 location + 10 recency. An unrelated hoodie scores 15 and is filtered at the 40 threshold.

"The same scoring runs in two places. The function is authoritative; the client copy powers the match-detail modal and fills in when the function hasn't run."

Expect: "Why not embeddings / ML?" → No labelled campus data, and an explainable score matters more here: the UI shows the user exactly which features matched. A model would be a black box for a marginal gain.

3d. A bug worth volunteering

`web/src/lib/ai.js`

"Our first AI search returned a water bottle for 'I lost my ID card'. Two causes: the word lost was matching the item's type field, which is literally 'lost'; and the tokenizer dropped 'id' for being two characters. We now strip intent words, require word-boundary matches on short tokens, and rank by weighted hits instead of taking the first match."

Volunteering a bug you found and fixed reads far better than claiming none.

Others you can offer:

- `suggestMatches` sent duplicate notifications — it writes `status`, which was also one of its own match inputs, so its write re-triggered it.
- The mock-data fallback latched permanently once the first snapshot timed out.
- The chat on your own listing opened a thread with yourself, because the counterparty was always "the poster" — exactly wrong for the seller trying to reach their buyer.

3e. The best security answer you have

`firestore.rules`, `web/src/lib/feed.js`

"We were judging claims on proof only the true owner should know — but the item's own description named those same marks and sat on a public document. A fraudster could read 'MEERA scratched on the back' off the card and hand it straight back. Hiding it in the UI would have been theatre: rules authorise whole documents, so anything on the card is readable over the REST API."

"So we split the document. The description moved to a `private/detail` subdocument only the poster and moderators can read — the same shape we already used for claim proofs."

Then volunteer the cost, because an examiner will look for it:

"That broke matching, because our keywords were derived from the description — publishing them would have leaked the same words as a list. Public tags are now the title plus a closed vocabulary of generic terms. The pairs still score 74–87% against a threshold of 50, but it is weaker, and it has to be: you cannot hide text and score on it in the browser, because whatever the matcher compares is readable by whoever runs the matcher. Doing it properly needs trusted compute — which is exactly why `suggestMatches` is a Cloud Function."

That last sentence is the one to land. It shows the constraint was understood rather than worked around.

4. Questions to rehearse

Architecture

- Why Firestore over PostgreSQL? → Realtime subscriptions, offline cache, rules as declarative authorization. Cost: denormalization, no joins, per-read billing.
- Which patterns and where? → Observer (`onSnapshot`), Strategy (AI dual path), Saga (two-party handshake), Chain of Responsibility (moderation), Repository (`lib/` data layer), State machine (item status).
- What would you change? → The 700 kB bundle has no code splitting.

Security

- How do you stop trust inflation? → `keeps()` guards; see 3a.
- Is the campus domain check secure? → The `hd` hint is not. The server-side re-check after sign-in is. Mismatched accounts are signed straight back out.
- What was your worst security bug? → `users` had `allow read: if true` while profiles store campus emails, so the whole user list was publicly enumerable over the REST API. Caught it by probing the endpoint unauthenticated.

Testing

- How do you know the backend works? → Firebase emulator suite; 26 assertions across all eight functions — matching, claims, reviews, transactions, flags, role gating.
- What isn't tested? → No frontend unit tests. Manual and emulator-driven only.

Process

- How did you split work? → See section 0. Answer honestly.
- Biggest disagreement? → Have a real one ready.

5. If a feature doesn't work live

Say what should happen, why it doesn't in this deployment, and offer evidence:

"Cloud Functions v2 needs the Blaze billing plan and we didn't enable billing on a student project. The functions are written and tested against the emulator — I can show you that run."

That is a deployment constraint, not a design failure, and examiners accept it. What they do not accept is a feature being presented as working when it is not.